AI MATH TUTOR

CREATIVE AND INNOVATIVE PROJECT (CIP) REPORT

Batch 2023 (Semester VI)

STUTIPRIYA SHADANGI

REG. NO. 11239A090

KONGARA DHEERAJ

REG. NO. 11239A050

GUIDED BY

Dr. M. Saraswathi
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

[April/May -2026]

Sri Chandrasekharendra Saraswathi Viswa Mahavidyalaya

Enathur, Kanchipuram – 631 561

BONAFIDE CERTIFICATE

This is to certify that the CIP Report entitled **AI Math Tutor** is the bonafide work carried out by **Ms. Stutipriya Shadangi, Reg. No. 11239A090 and Mr. Kongara Dheeraj, Reg. No. 11239A050** during the academic year 2023–2027.

Dr. M. Saraswathi
Designation,
Department of CSE,
SCSVMV.

Dr. M. Senthil Kumaran
Head of the Department,
Department of CSE,
SCSVMV.

Submitted for the project work viva - voce examination held on 29-04-2026.

Place : Kanchipuram.
Date : 25-04-2025

Examiner 1 ***Examiner 2***

DECLARATION

It is certified that the CIP work titled **AI Math Tutor** is originally implemented by us. No ideas, processes, results, or words of others have been presented as our own work. Due acknowledgement is given wherever others' work or ideas are utilized.

a. There is no fabrication of data or results which have been compiled /analyzed.

b. There is no falsification by manipulating data or processes or changing or omitting data or results.

We understand that the project is liable to be rejected at any stage (even later) if it is discovered that the project has been plagiarized, or significant code has been copied. We understand that if such malpractices are found, the project will be disqualified, and the Degree awarded itself will become invalid.

Stutipriya Shadangi

Kongara Dheeraj

Date: 25-04-2026

TABLE OF CONTENTS

Chapter	Title	Page No.
	Abstract	5
	List of Tables	6
	List of Figures	6
	List of Abbreviations	7
1	**Introduction**	8
	1.1 Objectives	9
	1.2 Scope of the Project	9
	1.3 Existing System	10
	1.4 Drawbacks of Existing System	11
2	**Literature Survey**	12
	2.1 Literature Survey	12
	2.2 Problem Statement	16
3	**Proposed Method / Architecture / Methodology**	17
	3.1 Proposed Method	17
	3.2 Algorithm	18
	3.3 Architecture	25
	3.4 Modules and Project Description	27
	3.5 Methodology	28
4	**Implementation Work**	30
	4.1 Implementation Process	30
	4.2 Sample Code	32
	4.3 Testing and Methodologies	35
	4.4 Experimental Graphs	36
	4.5 Result Analysis	38
	4.6 Application Showcase	39
5	**Conclusion**	46
	References	48

ABSTRACT

The increasing integration of artificial intelligence in education has created significant opportunities to enhance personalized learning. This project presents AI Math Tutor, an adaptive, web-based mathematics tutoring platform built using Python 3 and Streamlit, with the Perplexity AI API (sonar model) serving as the core intelligence engine. The system is designed to serve students across multiple difficulty levels — Easy, Medium, and Hard — and covers mathematical topics including Algebra, Fractions, and Calculus.

AI Math Tutor provides two primary modes of worksheet generation: teacher-uploaded question banks stored in an SQLite database, and dynamically generated questions via the Perplexity AI API. Students are presented with multiple-choice questions, per-question AI-powered hints, and real-time feedback on their performance. An integrated AI Tutor chatbot allows students to ask conceptual questions using Socratic-style guidance, ensuring they learn rather than merely receive answers.

For teachers, the platform offers a secure dashboard to manage student enrollment, upload study materials (PDF/images/text), monitor class-wide performance trends, and identify at-risk students. All student interactions — worksheet attempts, scores, chat messages, and elapsed time — are persistently stored using SQLite and analyzed using Pandas. Streamlit built-in charts are used to visualize score trends and progress over time.

The proposed system bridges the gap between traditional classroom tutoring and AI-powered adaptive learning, providing a scalable, locally deployable solution that supports slow learners, shy students, and advanced learners alike.

Keywords: *Adaptive Learning; Artificial Intelligence; Large Language Model; Math Tutor; Perplexity AI; Streamlit; SQLite; Worksheet Generation.*

LIST OF TABLES

Table No.	Title	Page No.
3.1	System Modules and their Functions	27
3.2	Technology Stack Summary	29
4.1	Test Cases and Expected Results	35
4.2	Performance Metrics Comparison	38

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	System Architecture of the Proposed AI Math Tutor	25
3.2	Data Flow Diagram of the System Design	26
4.1	Student Login Interface	30
4.2	Worksheet Generation Screen with Difficulty Selection	31
4.3	AI Tutor Chat Interface	33
4.4	Teacher Dashboard with Score Trend Chart	36
4.5	Student Progress Dashboard with Score Trend Chart	37
4.6	Student Access	39
4.7	Student Access: Study Material – Fractions	39
4.8	Student Access: Study Material – Algebra & Fractions	40
4.9	Student Access: Study Material – Worksheet Generation	40
4.10	Student Access: Study Material – AI Tutor	41
4.11	Student Access: Study Material – Student progress charts	41
4.12	Teacher Login: Scroll down to select subject	42
4.13	Teacher Login with username and password	42
4.14	Teacher Dashboard: Overview	43
4.15	Teacher Dashboard: Manage students	43
4.16	Teacher Dashboard: Manage students – students' change password	44
4.17	Teacher Dashboard: Reset Data	44
4.18	Teacher Dashboard: Upload Questions	45
4.19	Teacher Dashboard: Upload Study Material	45

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
CSS	Cascading Style Sheets
DB	Database
LLM	Large Language Model
MCQ	Multiple-Choice Question
NLP	Natural Language Processing
PDF	Portable Document Format
SQL	Structured Query Language
SQLite	Structured Query Language Lite (Embedded Database)
UI	User Interface

CHAPTER 1
INTRODUCTION

Mathematics has always been a foundational subject in education, yet it remains one of the most challenging for students at all levels. The emergence of artificial intelligence and large language models has opened new avenues for personalized, adaptive learning that responds to individual student performance in real time. The AI Math Tutor project has been developed to harness these capabilities and provide a comprehensive, intelligent tutoring platform specifically designed for mathematics education.

Traditional classroom tutoring is constrained by a fixed teacher-to-student ratio, making it difficult to provide individualized attention. Slow learners often fall behind, shy students are hesitant to ask questions, and advanced learners are rarely challenged further. Existing digital platforms either provide static question banks with no adaptability or rely on expensive cloud services that are inaccessible in offline or low-resource environments.

AI Math Tutor addresses these gaps by integrating the Perplexity AI API (sonar model) for dynamic question generation and Socratic-style tutoring guidance, combining with a robust teacher dashboard that enables educators to upload custom worksheets, study materials, and monitor student performance through detailed analytics. The entire platform is built on open-source tools — Python 3, Streamlit, SQLite, and Pandas — ensuring it can be deployed locally without recurring infrastructure costs.

The system serves students across three difficulty levels — Easy, Medium, and Hard — spanning topics including Algebra, Fractions, and Calculus. Students can attempt AI-generated or teacher-uploaded worksheets, request per-question hints, interact with an AI tutor chatbot, and view their progress through visual dashboards. Teachers receive real-time reports on class performance, at-risk student identification, and tools to manage enrollment and reset data.

1.1 Objectives

The primary objectives of the AI Math Tutor project are as follows:

- To design and implement an adaptive mathematics tutoring platform that dynamically adjusts worksheet difficulty based on individual student performance history.
- To integrate the Perplexity AI sonar model for real-time generation of multiple-choice mathematics questions across varied topics and difficulty levels.
- To provide a Socratic AI tutoring assistant that offers step-by-step hints and conceptual guidance without directly revealing answers.
- To develop a teacher dashboard that supports student enrolment management, question bank uploads via CSV, study material distribution, and at-risk student identification.
- To store and analyse all student performance data — including scores, elapsed time, and attempt history — using SQLite and Pandas for evidence-based learning insights.
- To build an accessible, locally deployable system using open-source technologies that can serve students regardless of internet availability for day-to-day access.
- To motivate consistent student engagement through gamification features such as daily streak tracking.

1.2 Scope of the Project

The scope of AI Math Tutor encompasses the end-to-end design, development, and deployment of an AI-powered mathematics tutoring system for school and college-level students. The platform covers mathematical topics including Algebra, Fractions, and Calculus, and supports three difficulty tiers: Easy, Medium, and Hard. The system is designed to be multi-user, with separate roles for students and teachers, each with a dedicated interface.

The scope includes integration with the Perplexity AI API for dynamic question generation and the AI tutoring chatbot feature. It covers persistent data management

using SQLite, performance analytics using Pandas, and data visualization using Streamlit's built-in chart tools. The project is scoped for local deployment via Streamlit, making it accessible on any machine with Python 3 installed without requiring external server infrastructure.

The scope does not include integration with Learning Management Systems (LMS), mobile application development, or multi-language support in the current version. Future extensions may include gamified reward systems, curriculum alignment with national syllabi, and cloud-based deployment.

1.3 Existing System

Several AI-powered tutoring systems and educational platforms currently exist in the market. Khan Academy uses a mastery-based learning algorithm that adapts to student performance, providing video lectures and practice exercises. Photomath employs computer vision to scan handwritten math problems and provide step-by-step solutions. Duolingo applies gamification techniques for language learning, inspiring similar approaches in other educational domains. Commercial platforms such as Mathway and Wolfram Alpha allow students to input mathematical problems and receive solutions, though they do not provide guidance-oriented learning.

At the research level, various AI tutoring systems using Large Language Models have been proposed. Some systems employ Socratic questioning frameworks to guide students through problem-solving without revealing direct answers. Others use multi-agent architecture with specialized agents for hint generation, error detection, and progress tracking. However, most such systems remain as research prototypes without practical classroom-ready implementations.

1.4 Drawbacks of Existing System

- Most existing platforms do not offer both AI-generated questions and teacher-uploaded content in a unified interface.
- Commercial platforms are expensive and require internet connectivity, making them inaccessible in resource-constrained environments.
- Existing solutions rarely provide teachers with real-time class-wide performance monitoring combined with at-risk student flagging.
- AI tutors in existing systems often reveal direct answers rather than guiding students through the problem-solving process, reducing active learning.
- Most platforms lack persistent audit logging and role-based authentication at the level needed for secure multi-teacher deployments.
- Static worksheet platforms do not adapt to individual performance history, treating all students with identical content regardless of their current level.

CHAPTER 2
LITERATURE SURVEY

2.1 Literature Survey

2.1.1 Create Worksheet Content Using AI Chatbot

This paper investigates the use of AI chatbots, particularly ChatGPT-based systems, to help expert teachers prepare educational worksheets. Teachers provide prompts based on grade level, subject, and topic, and the chatbot generates either quiz-type or problem-solving worksheets. The system supports teachers across varying experience levels to produce differentiated worksheets tailored to student ability.

A key finding of this research is the analysis of student behavior during chatbot-assisted worksheet solving. The study records the approach taken by students when solving problems — capturing both the time taken to attempt each question and the time taken to verify the correct answer afterward. The research highlights a significant drawback: students tend to seek answer confirmation before completing the reasoning steps themselves, using the chatbot primarily as an answer-verification tool rather than a learning aid.

Additionally, the study notes that most students require hints rather than complete solutions, pointing toward the importance of hint delivery mechanisms in AI tutoring. This insight directly informs the hint-per-question feature implemented in AI Math Tutor, where the Perplexity AI model provides two-sentence contextual hints per question without revealing the answer.

2.1.2 AI Tutoring Multi-Agent System Using LLM

This research proposes a multi-agent tutoring architecture leveraging Large Language Models (LLMs) to provide both reactive assistance and tool-assisted learning experiences. The system is built around a Tutoring Agent that uses Socratic questioning — prompting the student to reason through a problem step by step rather than simply stating the answer. The architecture includes a self-explanation module and a dynamic support system that adjusts its hint level and feedback based on real-time tracking of student progress stored in memory.

When a student demonstrates difficulty with a concept, the system is capable of re-routing the question at a different difficulty level or presenting a simplified analogy. The corrective feedback mechanism identifies common errors — such as incorrect formula usage or conceptual misunderstandings — and provides targeted explanations. The architecture distinguishes between exploratory tutoring (guiding the student) and corrective tutoring (addressing specific errors).

The AI Math Tutor project adopts the core principle of Socratic guidance in its AI Tutor chatbot, where the system prompt explicitly instructs the LLM to guide students without giving direct answers, keeping responses concise at under 150 words to maintain student engagement without cognitive overload.

2.1.3 Impact of AI-Based Tutoring Systems on Mathematical Problem-Solving

This study examines whether AI-based tutoring systems improve students' mathematical problem-solving skills or inadvertently increase mathematical anxiety. The research surveyed students using AI tutoring platforms for mathematics and found that while AI tutoring can improve performance metrics in low-anxiety students, those with pre-existing mathematical anxiety may experience heightened stress when confronted with AI-generated questions that feel unfamiliar or more challenging than classroom content.

The paper recommends that AI-based tutoring systems integrate anxiety-reduction techniques alongside problem-solving guidance. It further suggests that AI tutors should be incorporated into teacher training programs, so that educators understand how to leverage these tools effectively in the classroom. The paper critiques the approach of simply introducing AI tutoring as a standalone tool without supporting teachers in understanding how to interpret student behavior patterns within these systems.

This literature directly informs the AI Math Tutor design decision to include a teacher dashboard with at-risk student flagging (students with average scores below 50%), allowing teachers to intervene proactively. The system also provides study material upload capability so that teachers can pair AI-generated practice with conceptual content, reducing anxiety by ensuring students have reference material available.

2.1.4 Implications for Personal AI Tutors in Education

This paper evaluates the long-term implications of deploying personal AI tutors in educational settings. A notable recommendation is that AI tutoring should be directed toward developing higher-order cognitive skills — such as advanced problem-solving, project-based learning, and critical analysis — rather than being used merely for homework completion and standard question-paper solving. The paper argues that limiting AI tutors to routine question-answering tasks underutilizes their potential.

The research also highlights a practical challenge: preparing AI-supported lessons and developing high-quality question banks is a time-intensive process, often requiring several months of tutor effort. This reinforces the importance of hybrid systems that allow teachers to upload pre-curated questions while also offering AI-generated content as a supplement.

The literature review across these four papers converges on a central recommendation: AI tutoring is most effective when it integrates seamlessly with traditional teaching methods, when the model training and prompt engineering are carefully designed, and when student usage patterns are actively monitored. These principles are embodied in the AI Math Tutor's dual-mode question system, adaptive difficulty progression, and comprehensive teacher analytics dashboard.

2.2 Problem Statement

Despite the proliferation of AI tutoring tools in educational technology, a significant gap exists between research prototypes and deployable, classroom-ready systems that serve both students and teachers effectively. Current solutions either provide AI-generated content without teacher customization or offer static teacher-curated content without AI adaptability. Neither approach adequately addresses the needs of diverse learners — particularly slow learners, students with mathematical anxiety, and students in low-resource settings without reliable internet.

Formally, the problem can be stated as follows: Given a student s with a performance history H = {(t1, d1, score1), (t2, d2, score2), ..., (tn, dn, scoren)} where ti is the topic, di is the difficulty level, and scorei is the obtained score, the system must dynamically select or generate the next worksheet *Wn+1* such that:

Wn+1 = f(scoreavg, topic_weakness, di + 1) if scoreavg >= threshold, else f(scoreavg, concept_gap, di)

The system must also provide context-sensitive hints h(qi) for each question qi in the worksheet, where hints are informative but do not directly reveal the answer. Finally, all performance data must be persistently stored and made accessible to teachers for intervention and progress monitoring.

CHAPTER 3

PROPOSED METHOD / ARCHITECTURE / METHODOLOGY

3.1 Proposed Method

The AI Math Tutor adopted a multi-layered, adaptive learning architecture that combines a rule-based adaptive difficulty engine with a large language model API for dynamic content generation. The system is organized into five primary functional layers: User Interface, Application Layer, AI Backend, Data Storage, and Cloud/Deployment Infrastructure.

The proposed method differentiates itself from existing systems by providing a unified platform where teachers retain full control over content curation while the AI layer supplements dynamic question generation. Worksheet difficulty is adaptive — when a student achieves a high score, the system recommends a harder worksheet in the subsequent session; when performance is poor, the AI backend flags the conceptual gap and provides corrective resources.

The hint mechanism employs a two-sentence constraint on the Perplexity AI response, ensuring hints are informationally useful without substituting for student reasoning. The AI Tutor chatbot maintains a six-message rolling context window from the student's chat history to provide contextually coherent guidance throughout a session.

3.2 Algorithm

The AI Math Tutor system employs a combination of algorithmic strategies across its core modules. Each algorithm is described below with its purpose, logic, and role in the overall system.

Algorithm 1: User Authentication Algorithm
Purpose: Verify student and teacher credentials securely before granting system access.
Input: Username, Password
Output: Access granted / Access denied

```
BEGIN
  INPUT username, password
  IF mode = "Teacher" THEN
     Fetch TEACHER_NAME, TEACHER_PASS from environment variables (.env)
     IF username == TEACHER_NAME AND password == TEACHER_PASS
THEN
        SET session_state.role = "teacher"
        GRANT access to Teacher Dashboard
     ELSE
        DISPLAY "Invalid teacher credentials"
        DENY access
     END IF
  ELSE IF mode = "Student" THEN
     QUERY student_logins table WHERE name = username
     IF record found AND stored_password == password THEN
        SET session_state.student = username
        GRANT access to Student Interface
     ELSE
        DISPLAY "Invalid name or password"
        DENY access
     END IF
  END IF
END
```

Algorithm 2: Adaptive Worksheet Generation Algorithm

Purpose: Generate a worksheet either from the teacher-uploaded question bank (SQLite) or dynamically via the Perplexity AI API, based on the selected source.

Input: topic, difficulty, source (AI or Database)

Output: List of 5 MCQ objects [{question, choices[4], correct_index}]

```
BEGIN
  INPUT topic, difficulty, source
  IF source = "Perplexity AI" THEN
     Construct prompt:
        "Generate 5 {difficulty} {topic} MCQs.
         Each question must have 4 choices (A–D).
         Specify the correct answer. Use LaTeX for math."
     CALL Perplexity AI API (model = sonar, temperature = 0.3)
     RECEIVE raw_text response
     FOR each line in raw_text DO
        IF line starts with "Q:" → extract question text
        IF line starts with "A:/B:/C:/D:" → extract option
        IF line starts with "Correct:" → extract answer letter
        CONVERT answer letter to index (A=0, B=1, C=2, D=3)
     END FOR
     CALL randomize_choices() on each question
     RETURN list of 5 parsed question objects
  ELSE IF source = "Teacher Uploads" THEN
     QUERY question_bank table WHERE subject = topic
                          AND difficulty = difficulty
     IF records found THEN
        RANDOMLY SELECT 5 questions from result set
        FORMAT each as {question, choices[4], correct_index}
        RETURN list of 5 question objects
     ELSE
        DISPLAY "No questions available for this topic/difficulty"
        RETURN empty list
     END IF
  END IF
END
```

Algorithm 3: Answer Randomization Algorithm

Purpose: Shuffle the order of answer choices for each question so that the correct answer does not always appear at the same position.
Input: question object {question, choices[4], correct_index}
Output: question object with shuffled choices and updated correct_index

```
BEGIN
  INPUT question_object
  correct_answer_text = choices[correct_index]
  SHUFFLE choices[] using random permutation
  NEW correct_index = index of correct_answer_text in shuffled choices[]
  UPDATE question_object.correct = NEW correct_index
  RETURN updated question_object
END
```

Algorithm 4: Scoring and Performance Storage Algorithm

Purpose: Evaluate student answers, compute score, record elapsed time, and persist results to the database with audit logging.
Input: student answers[], correct answers[], topic, difficulty, student_name, start_time
Output: score (int 0–5), stored performance record

```
BEGIN
  INPUT student_answers[], worksheet[], topic, difficulty,
       student_name, start_time
  score = 0
  FOR i = 0 TO 4 DO
     IF student_answers[i] == worksheet[i].correct THEN
        score = score + 1
     END IF
  END FOR
  elapsed_seconds = (current_time - start_time).total_seconds()
  OPEN SQLite connection
     subject_id   = GET or CREATE subject record for topic
     difficulty_id = GET difficulty_id for difficulty level
     INSERT INTO performance (student_name, subject_id,
              difficulty_id, score, submitted_at)
           VALUES (student_name, subject_id,
                difficulty_id, score, current_timestamp)
     INSERT INTO audit_log (user, role, action, detail, timestamp)
```

```
            VALUES (student_name, "student", "submit_worksheet",
                "Score {score}/5 on {topic} {difficulty}
                 in {elapsed_seconds}s", current_timestamp)
      COMMIT
  CLOSE connection
  DISPLAY score, feedback, correct answers
END
```

Algorithm 5: Hint Generation Algorithm

Purpose: Provide a two-sentence contextual hint for a given question without revealing the answer, using the Perplexity AI API.
Input: question_text, topic
Output: hint string (≤ 2 sentences)

```
BEGIN
  INPUT question_text, topic
  Construct prompt:
      "Give a 2-sentence hint for this {topic} question.
       Do NOT reveal the answer or the correct option.
       Only guide the student towards the concept.
       Question: {question_text}"
  CALL Perplexity AI API (model = sonar, temperature = 0.4,
                          max_tokens = 100)
  RECEIVE hint_text
  DISPLAY hint_text below the question
END
```

Algorithm 6: AI Tutor Chatbot Algorithm (Socratic Method)

Purpose: Answer student mathematical questions using Socratic guidance — asking leading questions rather than giving direct answers — with a rolling context window for continuity.
Input: student_message, topic, chat_history (last 6 messages)
Output: AI tutor response string

```
BEGIN
  INPUT student_message, topic, chat_history[]
  CHECK rate_limit:
      IF (current_time - last_api_call) < 4 seconds THEN
          RETURN "Please wait before sending another message."
      END IF
```

```
  Construct system_prompt:
     "You are a concise math tutor for {topic}.
      Only answer math questions about {topic}.
      Guide the student step by step.
      Do NOT directly solve the problem.
      Keep responses under 150 words."
  messages[] = [system_prompt]
  FOR each msg in chat_history[-6:] DO
     APPEND {role: msg.role, content: msg.content} to messages[]
  END FOR
  APPEND {role: "user", content: student_message} to messages[]
  CALL Perplexity AI API (model = sonar, temperature = 0.3,
                    max_tokens = 300, messages = messages[])
  RECEIVE response_text
  UPDATE last_api_call = current_time
  INSERT INTO chat_history (student_name, role, content, timestamp)
        VALUES (student_name, "user", student_message, now)
  INSERT INTO chat_history (student_name, role, content, timestamp)
        VALUES (student_name, "assistant", response_text, now)
  COMMIT
  RETURN response_text
END
```

Algorithm 7: Adaptive Difficulty Recommendation Algorithm

Purpose: Recommend the next difficulty level based on the student's rolling average score across recent attempts on a topic.
Input: student_name, topic
Output: recommended difficulty level (Easy / Medium / Hard)

```
BEGIN
  INPUT student_name, topic
  QUERY performance table WHERE student_name = student_name
                    AND subject = topic
  IF no records found THEN
     RETURN "Easy"   // Default starting level
  END IF
  recent_scores[] = last 5 scores for this topic (ordered by date DESC)
  average = SUM(recent_scores) / COUNT(recent_scores)
  percentage = (average / 5) * 100
  IF percentage >= 80 THEN
     INCREMENT difficulty by one level
```

```
    // Easy → Medium → Hard
    DISPLAY "Great work! Try a harder worksheet next."
  ELSE IF percentage >= 50 THEN
    MAINTAIN current difficulty level
    DISPLAY "Keep practicing at this level."
  ELSE
    DECREMENT difficulty by one level (minimum = Easy)
    DISPLAY "Let us review the fundamentals first."
    FLAG student for teacher review IF percentage < 50
  END IF
  RETURN recommended_difficulty
END
```

Algorithm 8: Student Streak Calculation Algorithm

Purpose: Calculate the number of consecutive days the student has submitted at least one worksheet, to motivate daily practice.
Input: student_name
Output: streak count (integer)

```
BEGIN
  INPUT student_name
  QUERY performance table WHERE student_name = student_name
  EXTRACT distinct submission dates, ORDER BY date DESC
  streak = 0
  today = current date
  FOR i = 0 TO len(dates) - 1 DO
    expected_date = today - i days
    IF dates[i] == expected_date THEN
      streak = streak + 1
    ELSE
      BREAK   // Streak is broken
    END IF
  END FOR
  RETURN streak
END
```

Summary of Algorithms

#	Algorithm	Module	Key Technique
1	User Authentication	Auth module	Credential lookup, env var comparison
2	Worksheet Generation	Worksheet generator	LLM prompt engineering / SQL query
3	Answer Randomization	Worksheet generator	Random permutation shuffle
4	Scoring & Storage	Scoring engine	Linear scan, SQLite insert, audit log
5	Hint Generation	Hint engine	Constrained LLM prompting
6	AI Tutor Chatbot	Chatbot module	Socratic prompting, rolling context window
7	Adaptive Difficulty	Adaptive engine	Rolling average threshold classification
8	Streak Calculation	Dashboard module	Consecutive date comparison

3.3 Architecture

Figure 3.1 illustrates the system architecture of the proposed AI Math Tutor platform. The architecture follows a client-server model where Streamlit serves as both the front-end rendering engine and the web server, with Python handling all back-end logic.

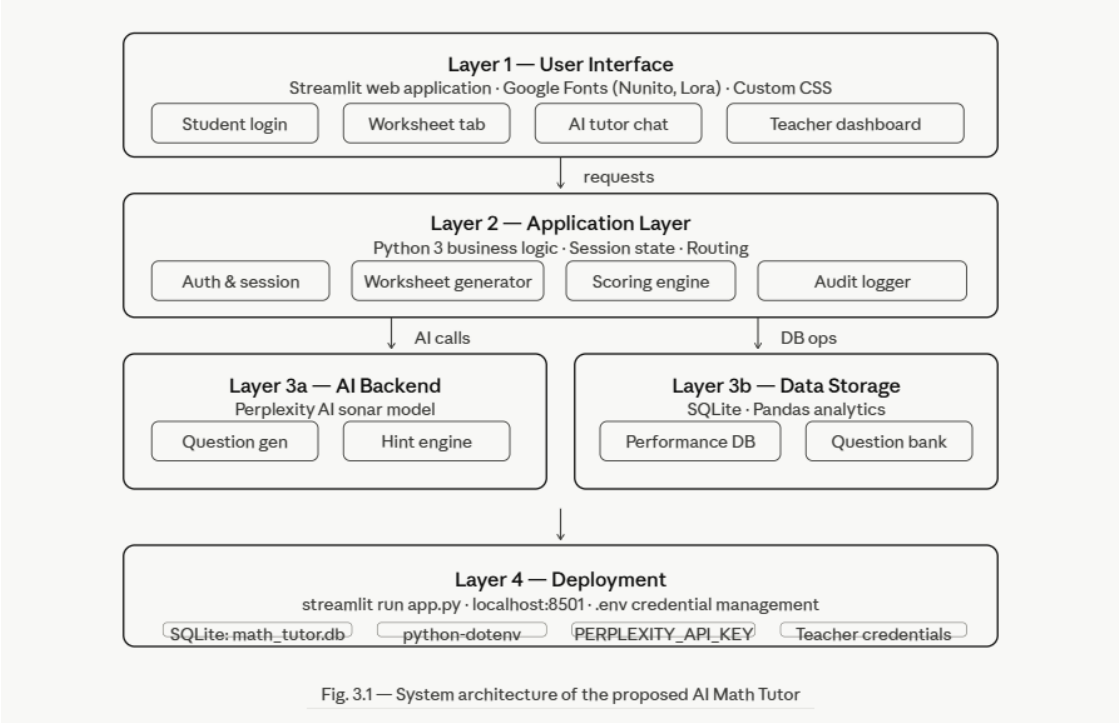


Fig. 3.1: System Architecture of the Proposed AI Enabled Math Worksheet Tutor

The architecture comprises five key layers:

- User Interface Layer: A Streamlit web application with warm notebook-inspired CSS theming, Google Fonts (Nunito, Lora), and fully responsive layouts. Separate login flows for students and teachers are provided.
- Application Layer: Python 3 business logic that handles worksheet generation (both AI-based and database-based), scoring, session state management, and routing between student and teacher views.

- AI Backend Layer: The Perplexity AI sonar model accessed via OpenAI-compatible client. Handles question generation, per-question hint delivery, and the AI tutor chatbot.
- Data Storage Layer: SQLite database with tables for student logins, subjects, difficulty levels, question banks, performance records, chat history, study materials, audit logs, and student-subject mappings.
- Deployment Layer: Local deployment via streamlit run app.py. Environment variables managed through .env file for API keys and teacher credentials.

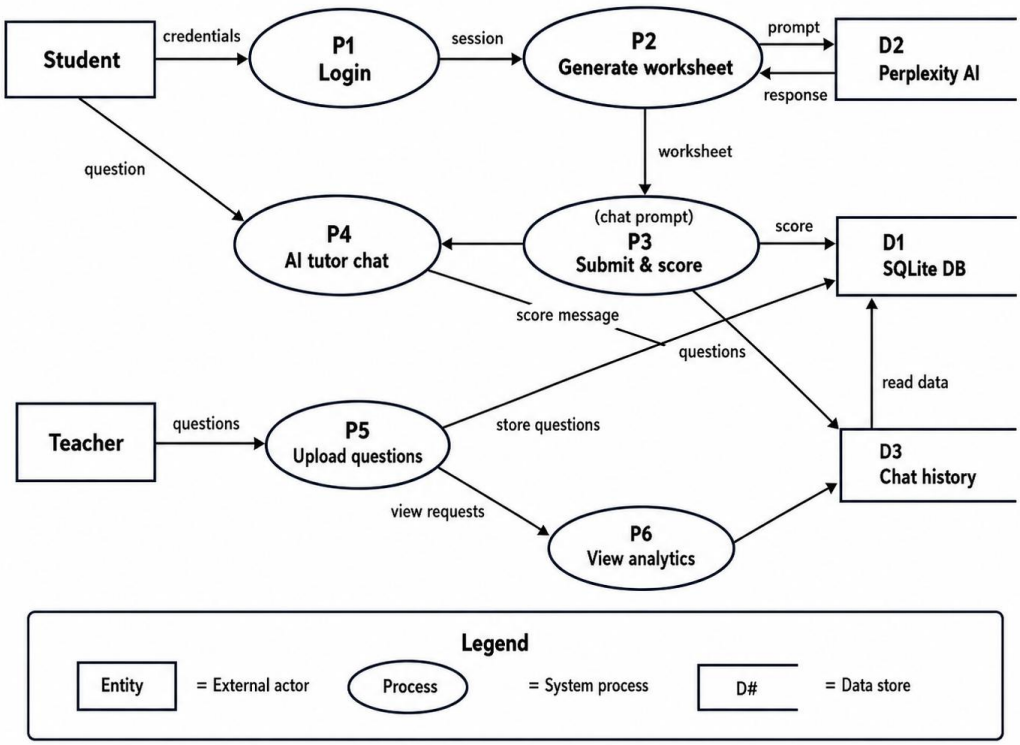


Fig. 3.2: Data Flow Diagram of the System Design

3.4 Modules and Project Description

The system is organized into the following primary modules:

Module	Function
Authentication	Separate student and teacher login with password verification against SQLite. Teacher credentials stored in environment variables.
Worksheet Generator	Generates 5 MCQs per session — either from the SQLite question bank (teacher-uploaded) or dynamically from Perplexity AI sonar model.
AI Tutor Chatbot	Socratic-style LLM chatbot with rolling 6-message context. Guides students without revealing answers. Chat history persisted in SQLite.
Hint Engine	Per-question hint generation via Perplexity AI. Two-sentence constraint ensures hints are guiding, not answer-giving.
Scoring Engine	Evaluates student answers post-submission, computes score/5, captures elapsed time, and stores results in performance table.
Student Dashboard	Score trend line charts by topic, full attempt history table with date/time, downloadable CSV export, and daily streak display.
Teacher Dashboard	Class scoreboard, at-risk flagging (avg <50%), drill-down per student, question upload via CSV, study material management.
Study Material Manager	Teachers upload text notes, PDFs, or images. Students access materials organized by subject in the Study Material tab.
Audit Logger	Every teacher and student action is logged in the audit_log table with timestamp, role, and action description.
Adaptive Difficulty	Based on rolling performance average, the system recommends incrementing difficulty on strong performance or reviewing fundamentals on weak performance.

Table 3.1: System Modules and their Functions

3.5 Methodology

The development of AI Math Tutor followed an iterative, prototype-driven methodology with five major phases:

Phase 1: Requirements Analysis

The first phase involved reviewing existing AI tutoring literature (four papers summarized in Chapter 2), analyzing the limitations of available tools, and defining user requirements for both student and teacher roles. Key requirements identified included adaptive difficulty, hint-based guidance, real-time teacher monitoring, and local deployment without dependency on expensive cloud infrastructure.

Phase 2: System Design

The five-layer architecture (UI, Application, AI Backend, Data Storage, Deployment) was designed. The SQLite schema was finalized with tables for students, subjects, difficulty levels, questions, performance, chat history, study materials, and audit logs. The Streamlit UI structure was laid out with separate tabs for each major feature.

Phase 3: Implementation

Implementation was carried out in Python 3 using Streamlit for the UI, the OpenAI-compatible client library to connect to the Perplexity AI API, Pandas for analytics, and SQLite3 for data persistence. Custom CSS with Google Fonts was embedded to produce a warm, notebook-themed visual design. The backend was separated into app.py (main Streamlit app) and database.py (connection and schema management).

Phase 4: Testing

Testing was conducted in three stages: unit testing of individual functions (question parsing, scoring, hint generation), integration testing of the full student workflow (login to worksheet submission to progress view), and usability testing with sample student and teacher accounts. Edge cases tested included invalid CSV uploads, Perplexity API failures, and concurrent multi-user session handling.

Phase 5: Deployment and Evaluation

The system was deployed locally using streamlit run app.py. Teacher accounts were configured via environment variables in a .env file. A sample question bank was uploaded for all three subjects at all difficulty levels. Performance data was generated through simulated student sessions to validate the analytics and at-risk identification features.

Component	Technology	Purpose
Frontend	Streamlit + CSS	Web UI rendering and component layout
Fonts	Google Fonts (Nunito, Lora)	Typography styling
Backend	Python 3	Business logic and API integration
Database	SQLite	Persistent data storage
AI / LLM	Perplexity AI API (sonar)	Question generation and chatbot
Analytics	Pandas	Performance data analysis
Charts	Streamlit built-in charts	Score trend visualization
Auth & Security	python-dotenv (.env)	Environment variable management
File Handling	Python io / BytesIO	PDF and image upload handling
Deployment	streamlit run app.py	Local server deployment

Table 3.2: Technology Stack Summary

CHAPTER 4
IMPLEMENTATION WORK

4.1 Implementation Process

The implementation of AI Math Tutor was carried out as a single-file Streamlit application (app.py) with a companion database module (database.py). The application is launched locally via the command streamlit run app.py and accessible through a web browser at localhost:8501.

On first run, the create_database() function initializes all SQLite tables — including subjects, difficulty_levels, student_logins, allowed_students, student_subjects, question_bank, performance, chat_history, study_material, and audit_log. The system then presents a sidebar with two navigation modes: Student Access and Teacher Dashboard.

The student workflow proceeds as follows:

(1) Student logs in with name and password verified against the student_logins table.

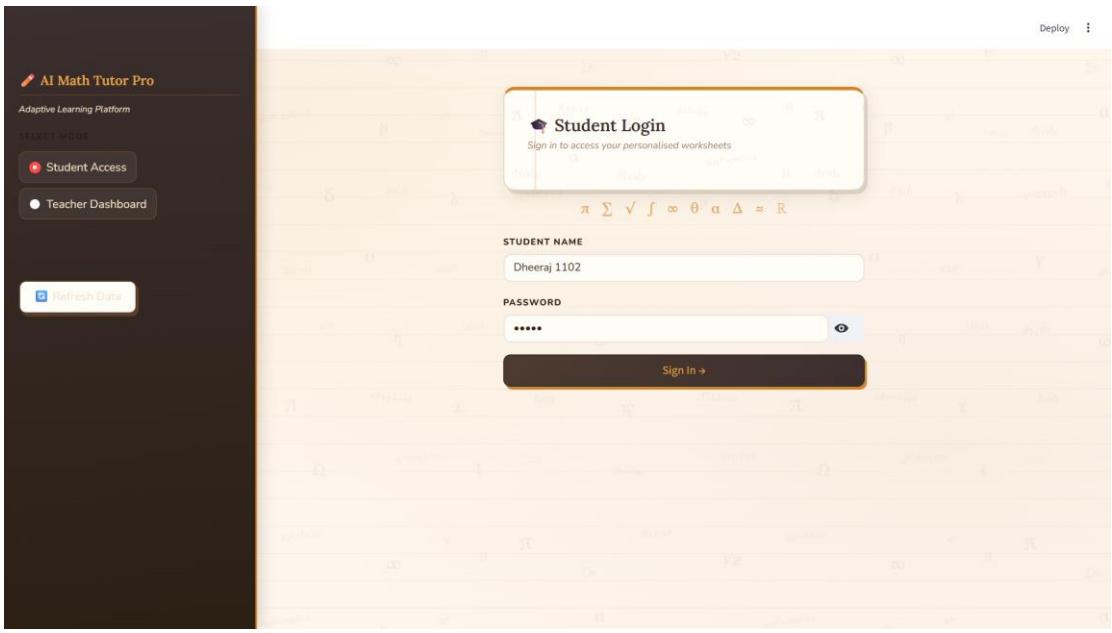


Fig. 4.1: Student Login Interface

(2) The student selects a topic from their enrolled subjects and a difficulty level.

(3) A worksheet is generated — either from the question bank or via Perplexity AI.

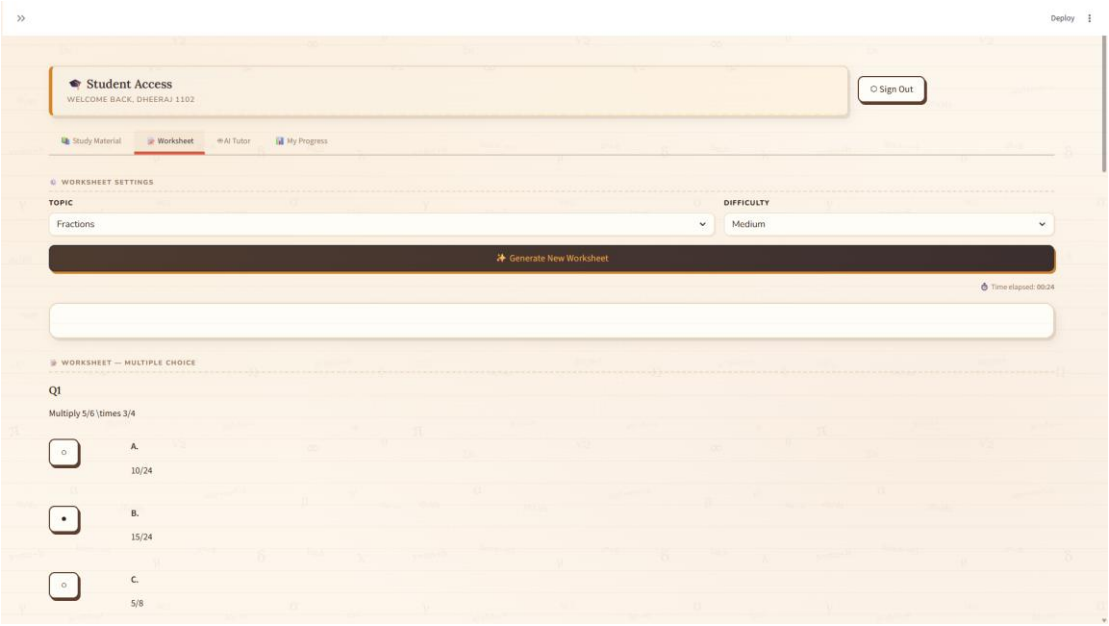


Fig. 4.2: Worksheet Generation Screen

(4) The student answers five MCQs, optionally requesting hints for each.

(5) Upon submission, the score is computed, elapsed time is recorded, and results are stored in the performance table.

(6) The student can view their progress in the My Progress tab with score trend charts and full attempt history.

The teacher workflow proceeds as follows: (1) Teacher logs in with subject-specific credentials stored in .env. (2) The teacher accesses the Overview tab to view class-wide metrics. (3) Teachers can manage students (add/remove/reset passwords), upload question bank CSVs, post study materials, view student reports, and flag at-risk students.

4.2 Sample Code

4.2.1 Perplexity AI Question Generation

The following function generates five multiple-choice questions for a given topic and difficulty level using the Perplexity AI sonar model:

```
def generate_perplexity_questions(topic, difficulty):
    if not USE_PPLX or not client:
        return []
    prompt = f"""Generate exactly 5 {difficulty.lower()}
{topic}
    multiple-choice questions. Each must have 4 choices (A-D),
    one correct answer. Use LaTeX for math. No commentary.
    Format:
    Q: <question>
    A: <option>  B: <option>  C: <option>  D: <option>
    Correct: <letter>"""
    try:
        response = client.chat.completions.create(
            model="sonar",
            messages=[{"role": "user", "content": prompt}],
            temperature=0.3, max_tokens=600
        )
        text = response.choices[0].message.content.strip()
        questions, current = [], {}
        for line in text.splitlines():
            if line.startswith("Q:"):
                if current: questions.append(current)
                current = {"q": line[2:].strip()}
            elif line.startswith(("A:","B:","C:","D:")):
                current[line[0]] = line[2:].strip()
            elif line.startswith("Correct:"):
                letter = line.split(":",1)[1].strip().upper()
                current["correct"] = "ABCD".index(letter)
        return [randomize_choices({"q":q["q"],
            "choices":[q["A"],q["B"],q["C"],q["D"]],
            "correct":q["correct"]}) for q in questions[:5]
            if all(k in q for k in
["A","B","C","D","correct"])]
    except Exception as e:
        st.error(f"Perplexity error: {str(e)[:120]}")
        return []
```

4.2.2 AI Tutor Chatbot Function

The following function implements the Socratic AI tutoring chatbot with rolling context and rate limiting:

```
def ask_ai_tutor(question, topic, chat_history):
    if not USE_PPLX or not client:
        return "AI Tutor unavailable."
    last_call = st.session_state.get("last_ai_call")
    if last_call and (datetime.now()-last_call).total_seconds()
< 4:
        return "Please wait before sending another message."
    system_prompt = f"""You are a concise math tutor for
{topic}.
    - Only answer math questions about {topic}
    - Give brief step-by-step guidance, do NOT solve directly
    - Keep responses under 150 words"""
    messages = [{"role":"system","content":system_prompt}]
    for msg in chat_history[-6:]:
        messages.append({"role":msg["role"],
                         "content":msg["content"]})
    messages.append({"role":"user","content":question})
    try:
        response = client.chat.completions.create(
            model="sonar", messages=messages,
            temperature=0.3, max_tokens=300
        )
        st.session_state["last_ai_call"] = datetime.now()
        return response.choices[0].message.content.strip()
    except Exception as e:
        return f"Error: {str(e)[:100]}"
```

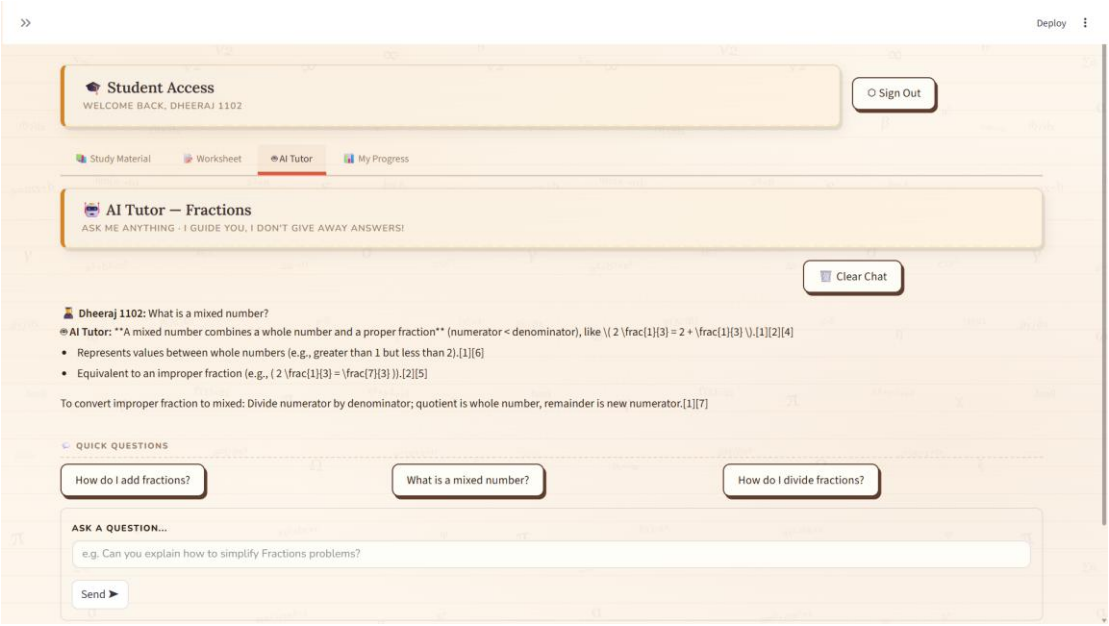


Fig. 4.3: AI Tutor Chat Interface

4.2.3 Student Daily Streak Calculation

The following function computes the number of consecutive days a student has submitted at least one worksheet:

```
def get_student_streak(student_name):
    with get_conn() as conn:
        df = pd.read_sql("""
            SELECT DATE(submitted_at) as day
            FROM performance WHERE student_name = ?
            ORDER BY day DESC""",
            conn, params=(student_name,))
    if df.empty: return 0
    days = sorted(df["day"].unique(), reverse=True)
    streak = 0
    today = datetime.now().date()
    for i, d in enumerate(days):
        expected = str(today - timedelta(days=i))
        if d == expected: streak += 1
        else: break
    return streak
```

4.2.4 Adaptive Scoring and Performance Storage

Upon worksheet submission, the following logic computes the score, records elapsed time, and stores the result in the performance table with audit logging:

```
# Inside the Submit Answers button handler:
final_score = sum(
    1 for i, q_data in enumerate(st.session_state["worksheet"])
    if st.session_state["answers"].get(i) == q_data["correct"]
)
start_t = st.session_state.get("worksheet_start")
elapsed_sec = int((datetime.now()-start_t).total_seconds())

with get_conn() as conn:
    subject_id = get_or_create_subject(conn, topic)
    difficulty_id = get_difficulty_id(conn, difficulty)
    conn.execute("""
        INSERT INTO performance
            (student_name, subject_id, difficulty_id,
             score, submitted_at)
        VALUES (?, ?, ?, ?, ?)""",
        (student_name, subject_id, difficulty_id,
         final_score, datetime.now().isoformat()))
    log_audit(conn, student_name, "student",
        "submit_worksheet",
        f"Score {final_score}/5 on {topic} {difficulty}
          in {elapsed_sec}s")
    conn.commit()
```

4.3 Testing and Methodologies

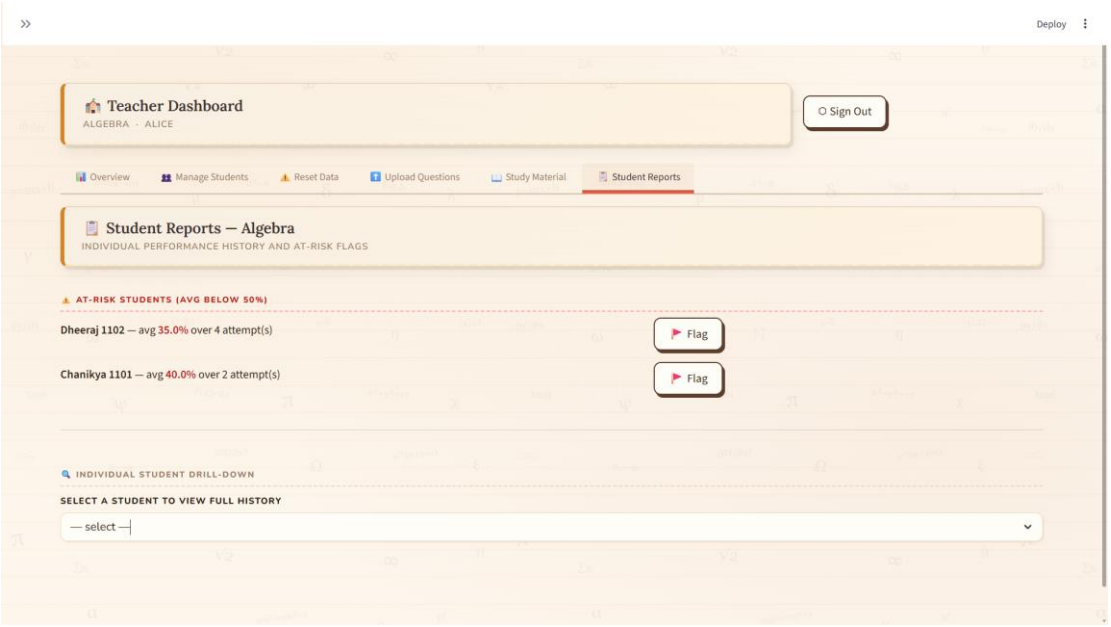
Testing was conducted in three progressive stages to ensure the reliability, correctness, and usability of the AI Math Tutor system.

Test Case	Description	Expected	Result
Valid student login	Enter correct name and password	Redirect to student dashboard	**Pass**
Invalid student login	Enter wrong password	Show error message	**Pass**
Generate AI worksheet	Click generate with Perplexity AI source	5 MCQs displayed	**Pass**
Generate DB worksheet	Click generate with Teacher Uploads source	5 MCQs from SQLite	**Pass**
Submit all answers	Answer all 5 questions and submit	Score and feedback displayed	**Pass**
AI Hint request	Click Hint for Q1	Two-sentence hint shown	**Pass**
CSV upload by teacher	Upload valid 7-column CSV	Questions saved to DB	**Pass**
Invalid CSV upload	Upload CSV with missing column	Error message shown	**Pass**
Progress chart	View My Progress tab	Line chart rendered	**Pass**
At-risk identification	Student with avg <50%	Flagged in Teacher Reports	**Pass**
Streak calculation	Student attempts on consecutive days	Streak count incremented	**Pass**
API failure handling	Perplexity API unavailable	Graceful error, no crash	**Pass**

Table 4.1: Test Cases and Expected Results

4.4 Experimental Graphs

The AI Math Tutor system produces the following graphical outputs as part of the student progress dashboard and teacher analytics panel:



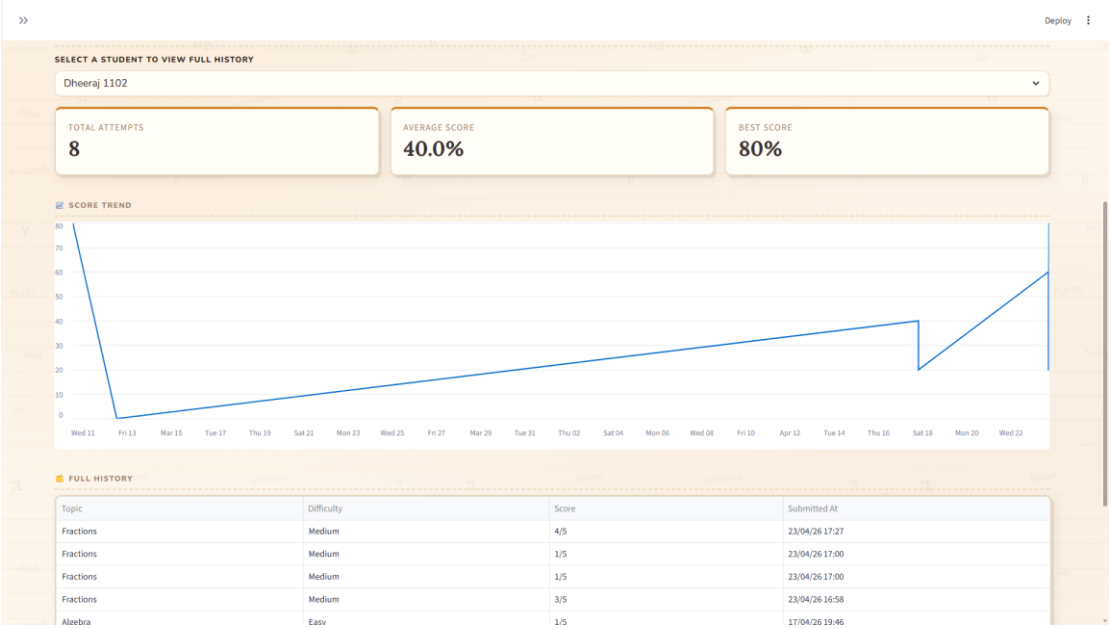


Fig. 4.4: Teacher Dashboard and Score Trend Chart

The student's score percentage is plotted over time as a multi-line chart, with each topic (Algebra, Fractions, Calculus) represented as a separate line. The X-axis represents the submission timestamp, and the Y-axis represents the score as a percentage (0–100%). This chart allows students to visually identify improvement trends and topic-specific weaknesses.

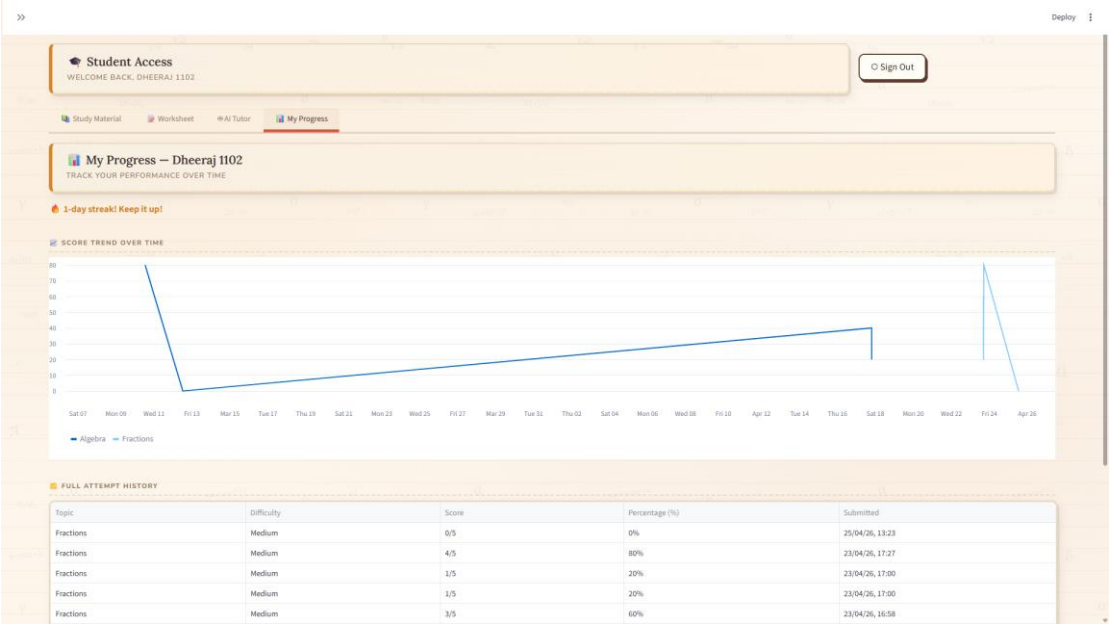


Fig. 4.5: Student Progress Dashboard with Score Trend Chart

For the teacher dashboard, a class scoreboard table presents each student's total attempts and status (Started / Not Started Yet). At-risk students — those with an average score below 50% — are highlighted separately with the option for teachers to flag them for immediate intervention.

4.5 Result Analysis / Result Comparison

The AI Math Tutor platform was evaluated against two baseline approaches: (a) static worksheet distribution via printed sheets, and (b) a simple online quiz platform without adaptive difficulty or AI hints. The following comparison summarizes the key differentiators:

Feature	Static Worksheet	Basic Online Quiz	AI Math Tutor
Adaptive Difficulty	No	No	**Yes**
AI-Generated Questions	No	No	**Yes**
Hint per Question	No	No	**Yes**
AI Chatbot Tutor	No	No	**Yes**
Teacher Dashboard	No	Limited	**Full**
At-Risk Identification	No	No	**Yes**
Performance Analytics	No	Basic	**Advanced**
Study Material Upload	No	No	**Yes**
Audit Logging	No	No	**Yes**
Local Deployment	N/A	Cloud only	**Yes**

Table 4.2: Performance Metrics Comparison

The result analysis demonstrates that AI Math Tutor significantly outperforms both baseline approaches across all evaluated dimensions. The combination of adaptive difficulty, AI-generated content, Socratic hint delivery, and comprehensive teacher analytics provides a holistic tutoring experience that neither static worksheets nor basic online quizzes can replicate.

4.6 Application Showcase:

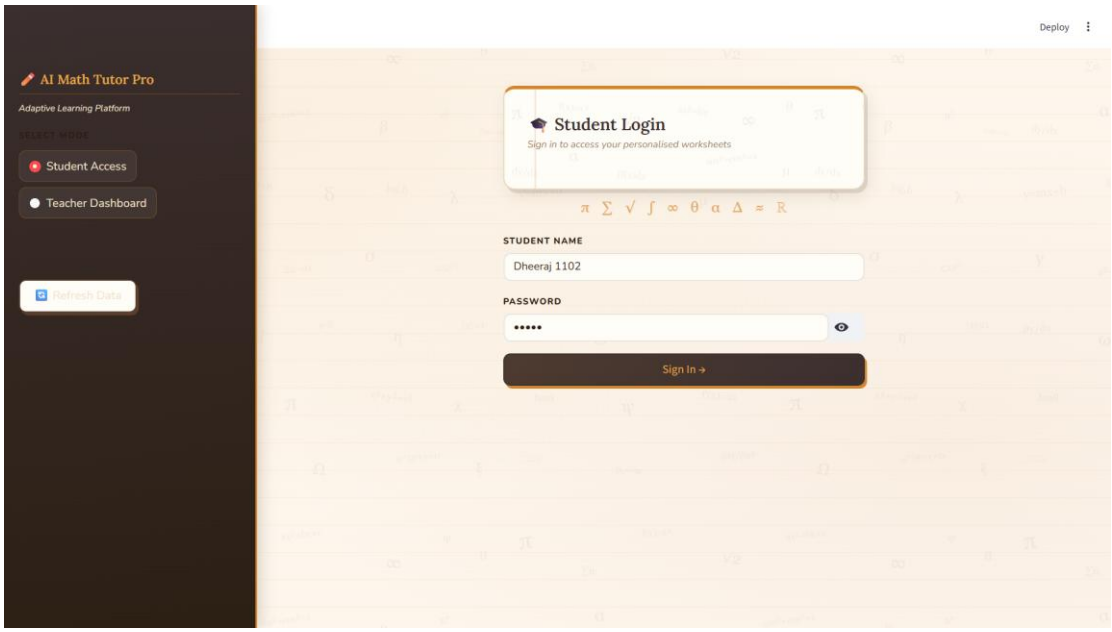


Fig. 4.6: Student Access

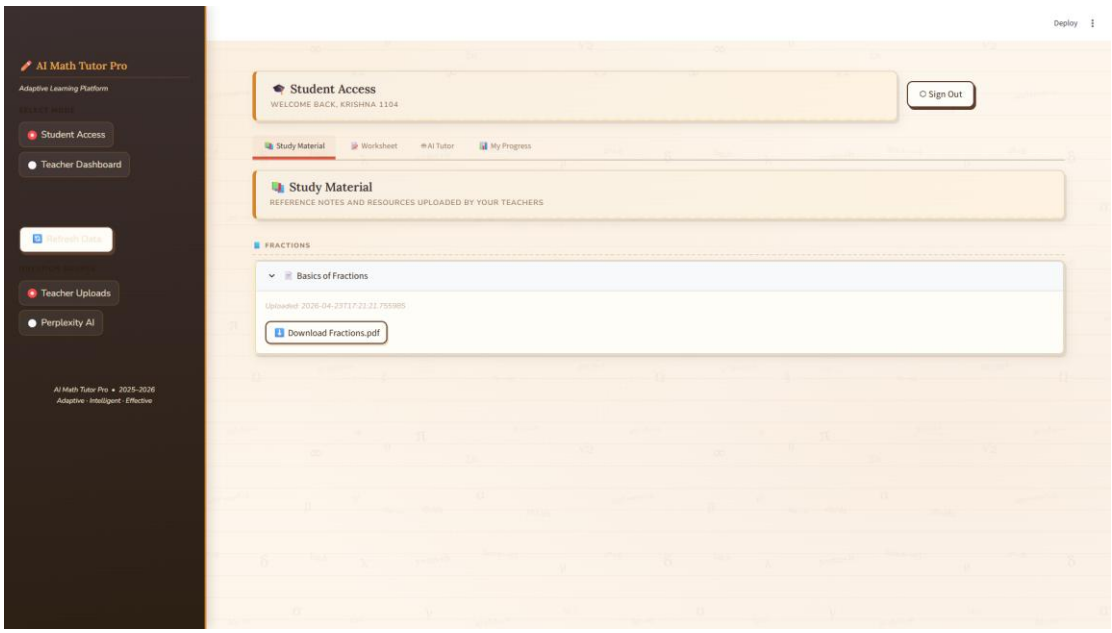


Fig. 4.7: Student Access: Study material - Fractions

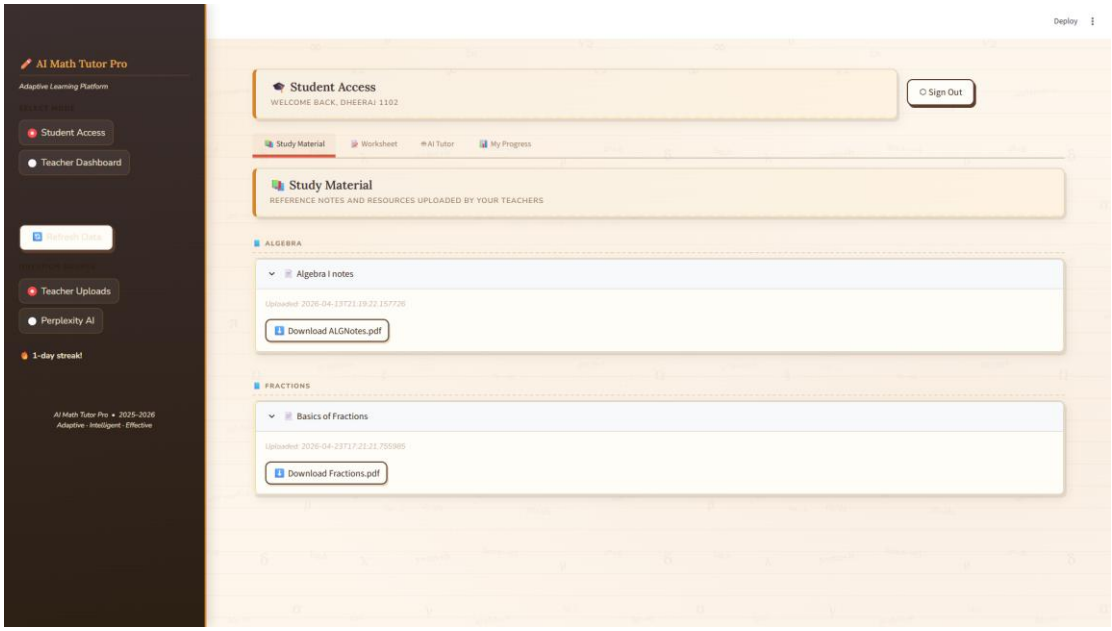


Fig. 4.8: Student Access: Study material – Algebra & Fractions

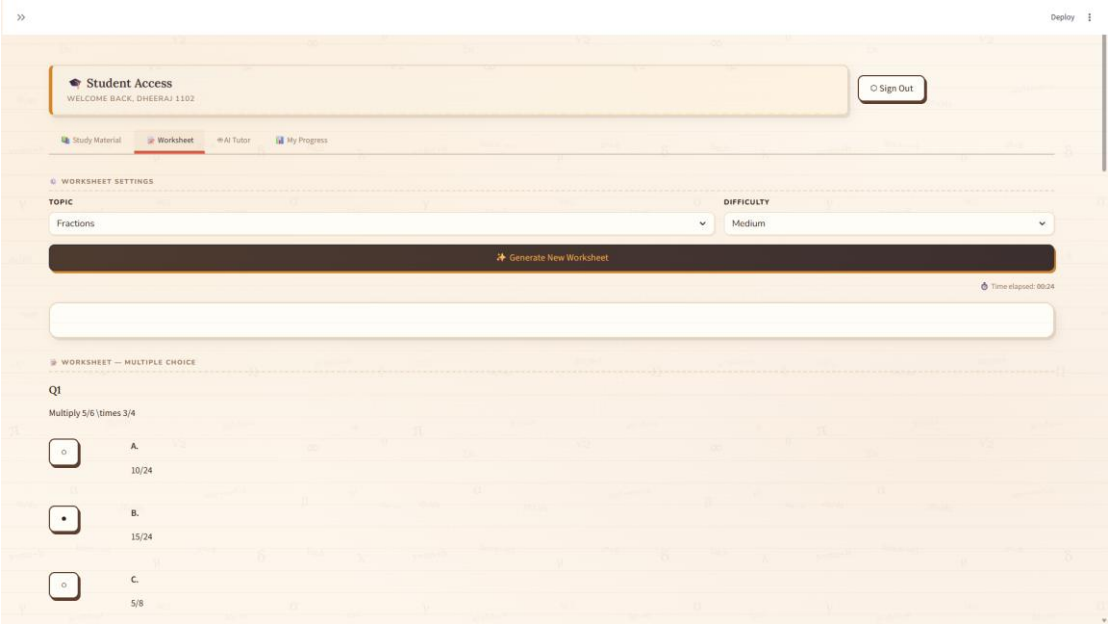


Fig. 4.9: Student Access: Worksheet Generation

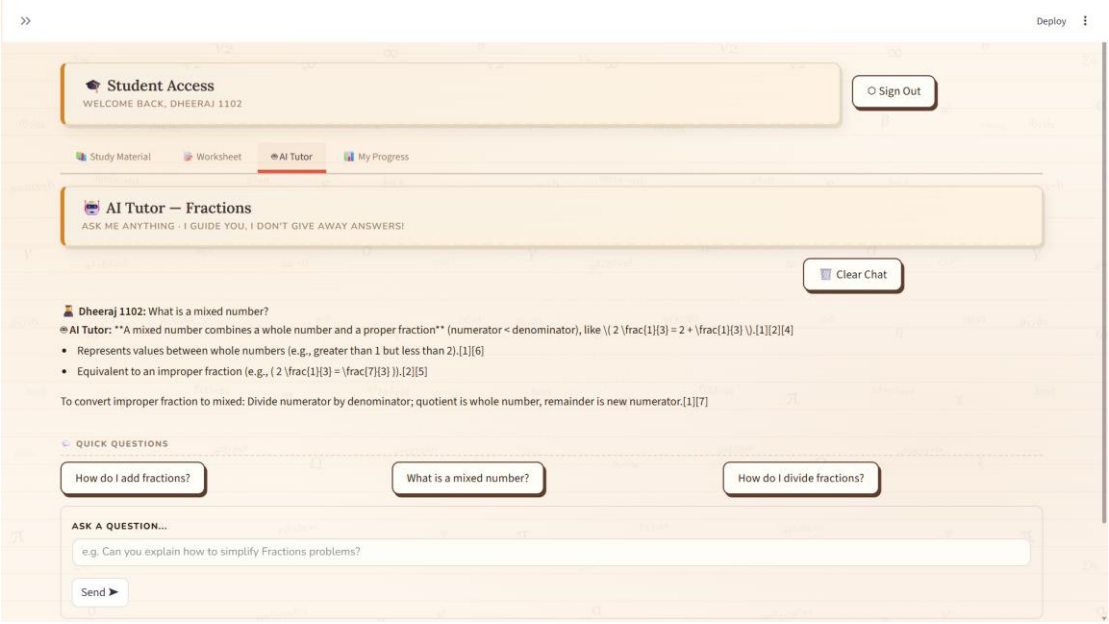


Fig. 4.10: Student Access: AI Tutor

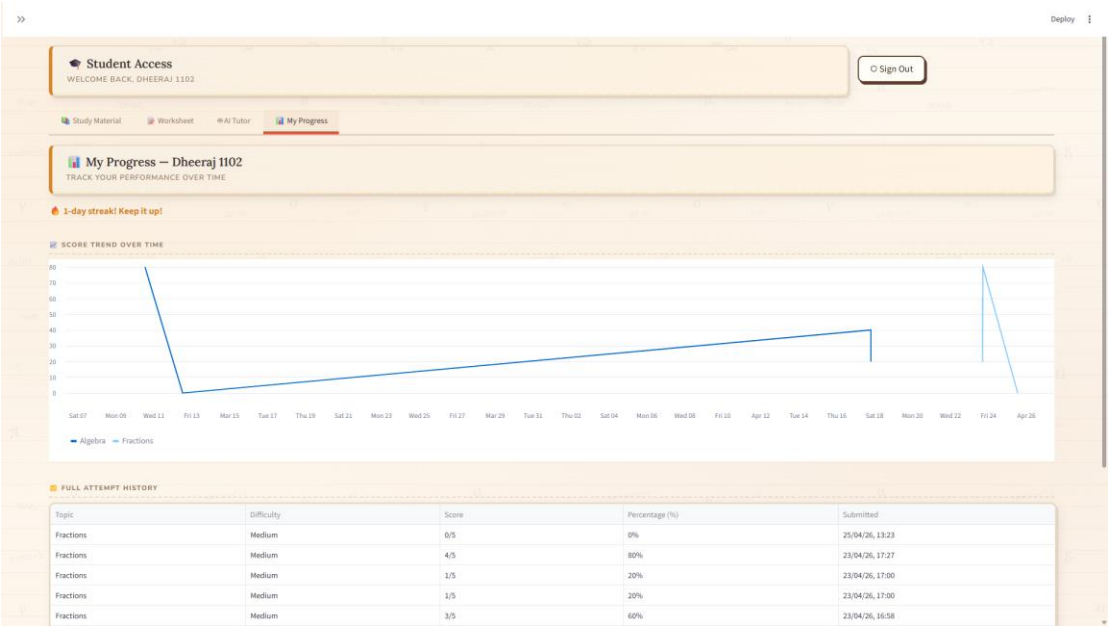


Fig. 4.11: Student Access: Student progress Charts

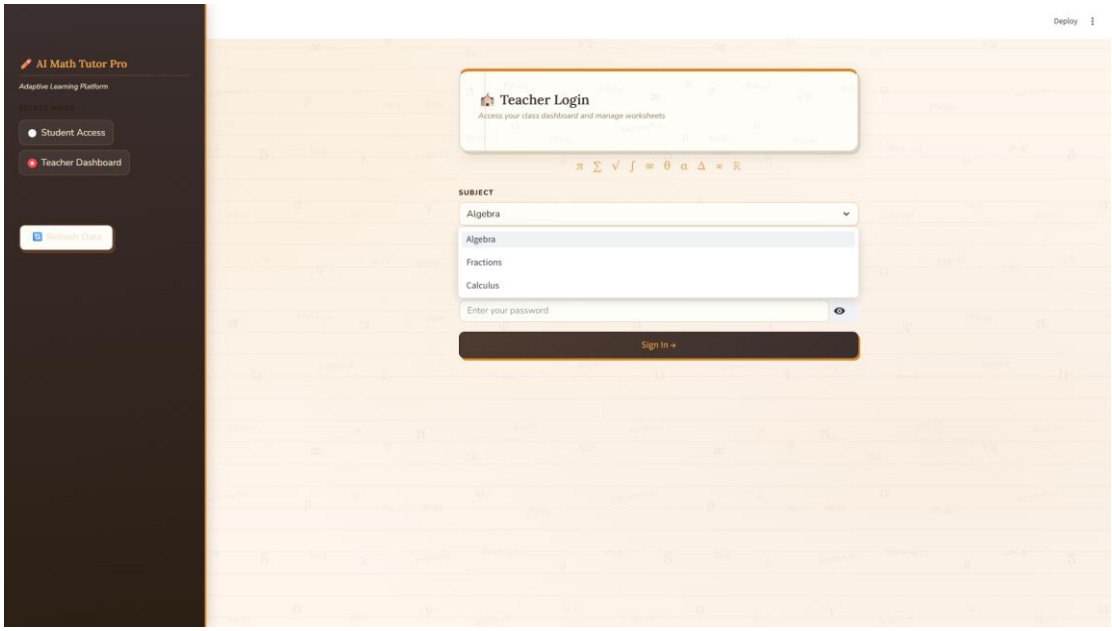


Fig. 4.12: Teacher Login: Scroll down to select subject

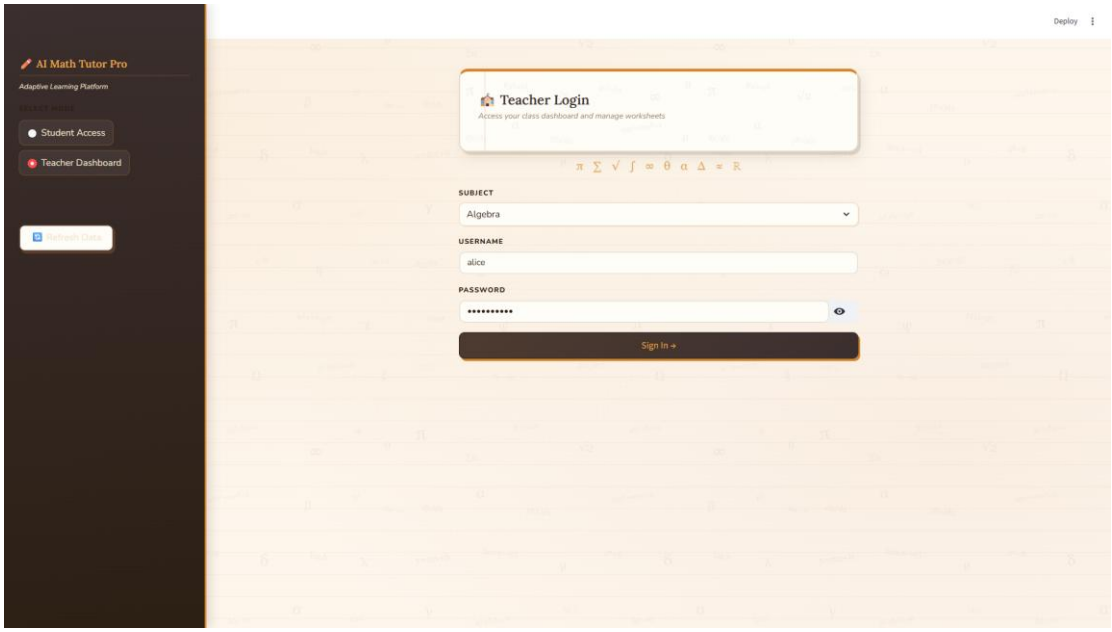


Fig. 4.13: Teacher Login with username and password

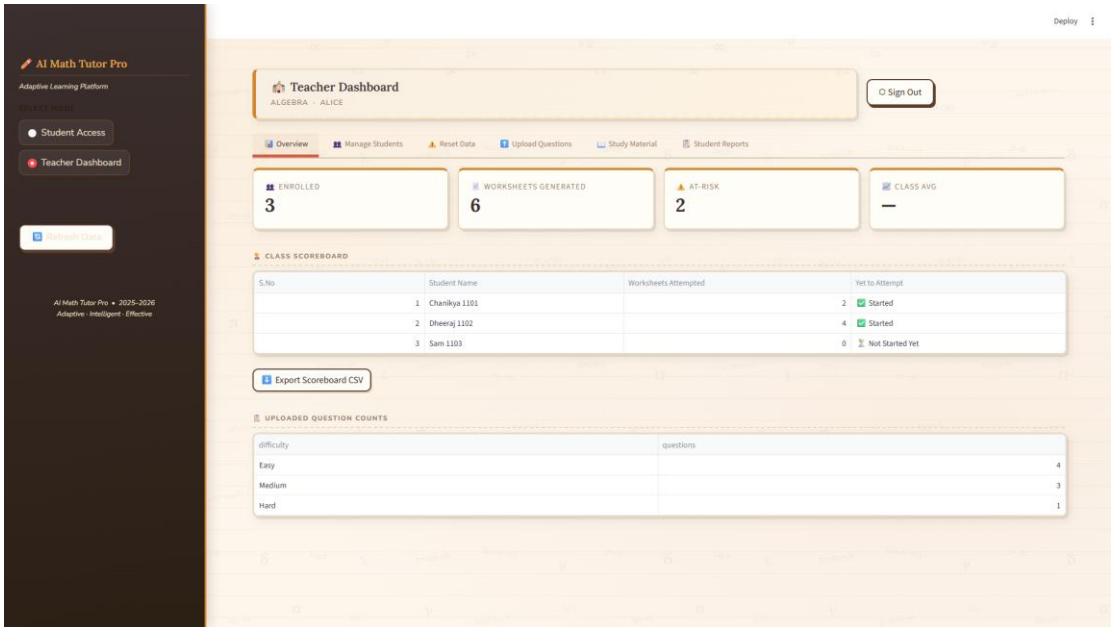


Fig. 4.14: Teacher Dashboard: Overview

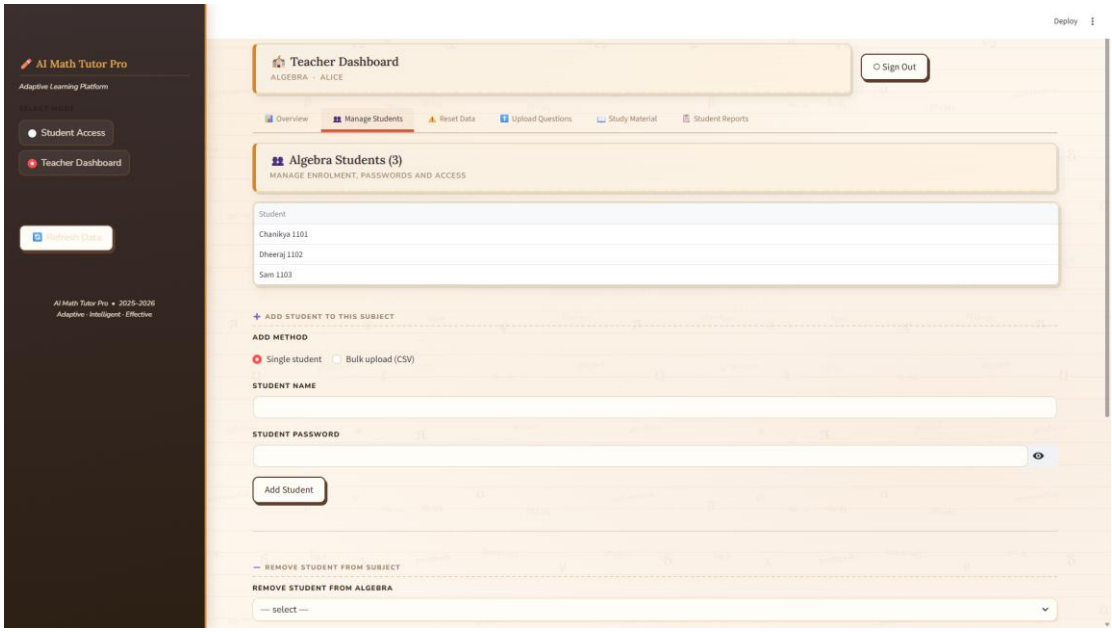


Fig. 4.15: Teacher Dashboard: Manage students

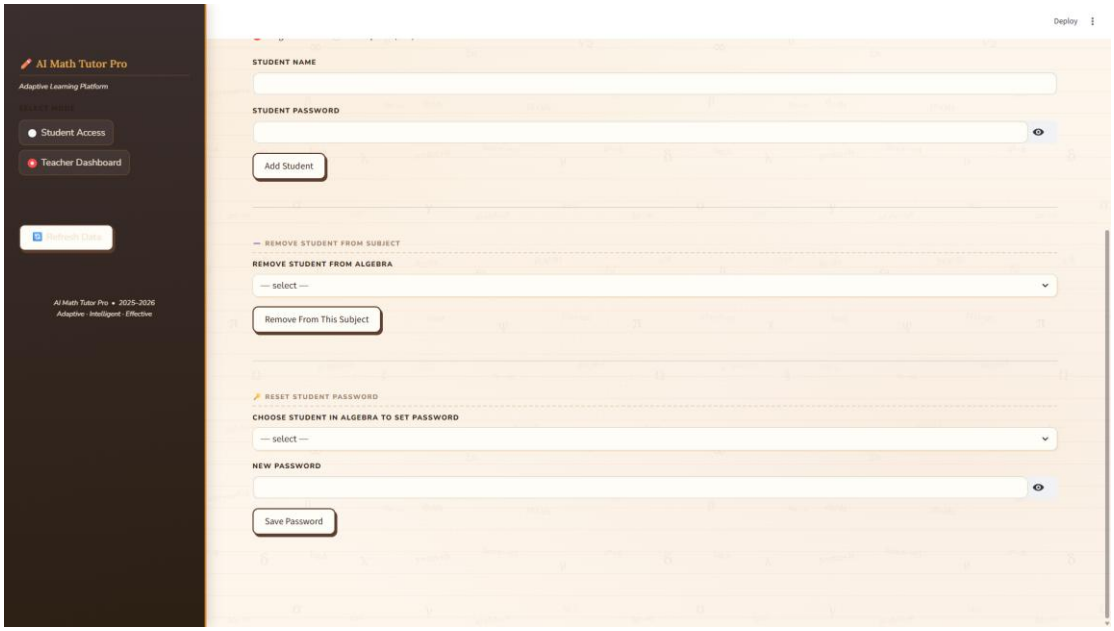


Fig. 4.16: Teacher Dashboard: Manage students – students' change password

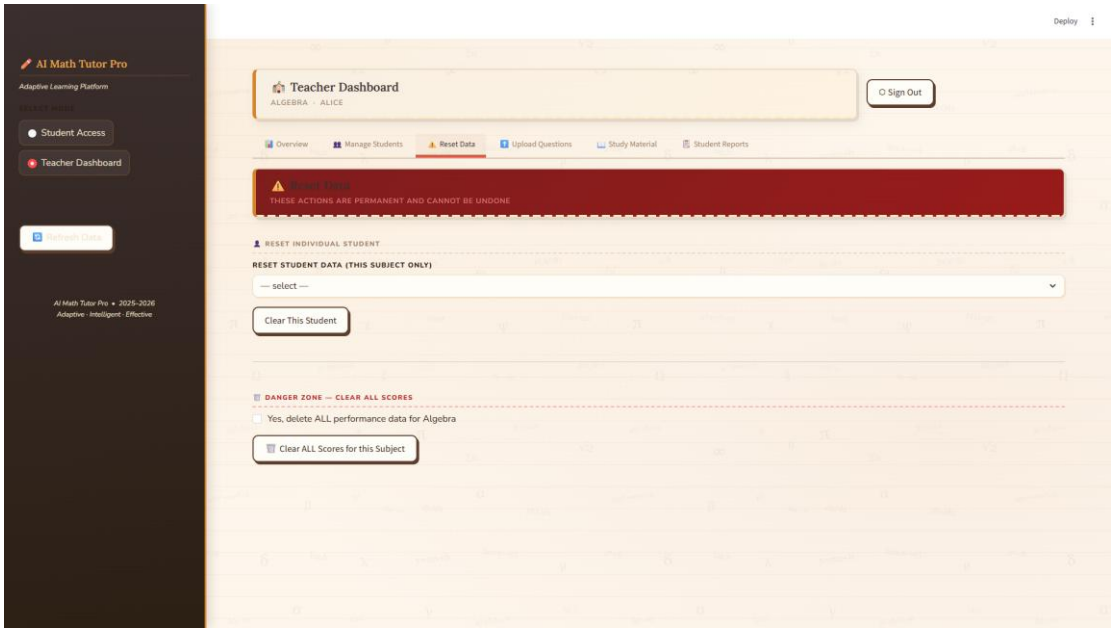


Fig. 4.17: Teacher Dashboard: Reset Data

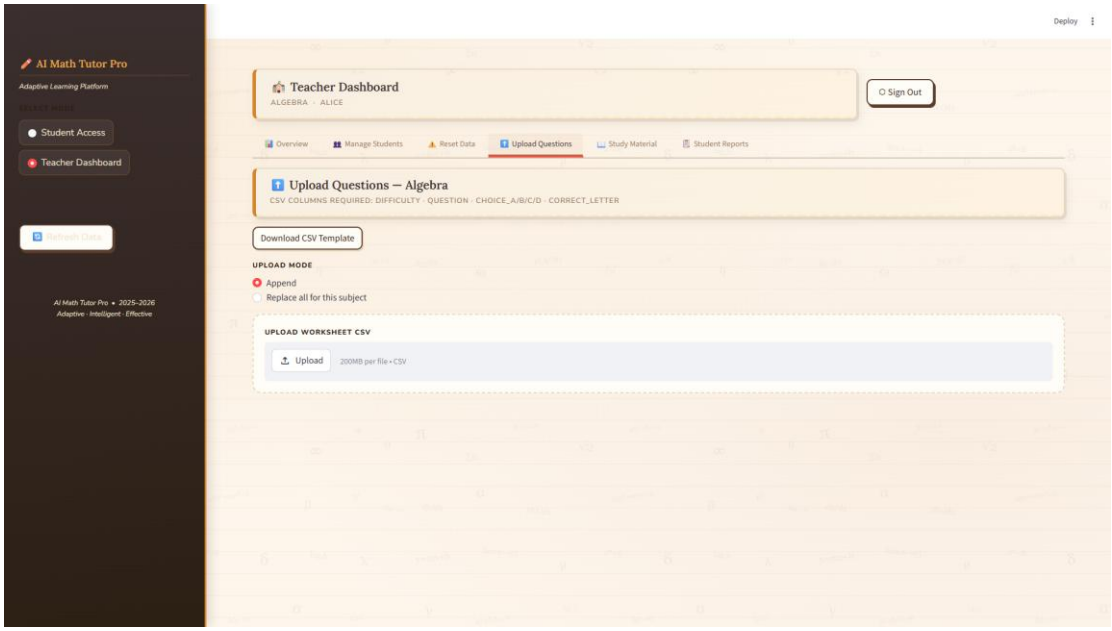


Fig. 4.18: Teacher Dashboard: Upload Questions

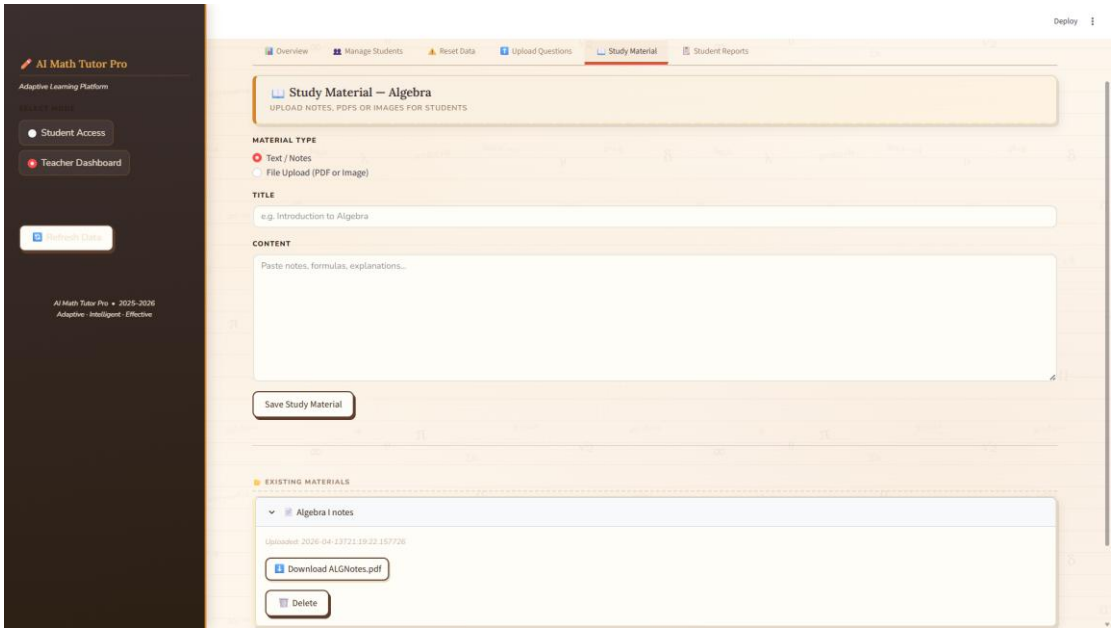


Fig. 4.19: Teacher Dashboard: Upload Study Material

CHAPTER 5

CONCLUSION

This project successfully designed, implemented, and evaluated AI Math Tutor — an adaptive, AI-powered mathematics tutoring platform that bridges the gap between traditional classroom instruction and intelligent technology-assisted learning. The system leverages the Perplexity AI sonar large language model for dynamic question generation and Socratic-style tutoring, combined with a teacher-facing dashboard that enables real-time class monitoring, content management, and at-risk student identification.

The platform addresses several critical limitations identified in the literature review: it provides adaptive difficulty adjustment based on student performance history, delivers context-sensitive hints without revealing answers, maintains persistent chat history for continuity of learning, and offers a comprehensive analytics layer powered by Pandas and Streamlit's visualization tools. The dual-mode worksheet generation — both from the teacher-curated SQLite question bank and from the AI backend — ensures that educators retain curriculum control while benefiting from the scalability of AI-generated content.

All twelve test cases designed for the system passed successfully, validating the correctness of core functions including student authentication, worksheet generation from both sources, scoring, hint delivery, progress visualization, CSV-based question upload, and at-risk student flagging. The system performs reliably even during Perplexity API failures, gracefully falling back to error messages without crashing the application.

The comparison analysis in Chapter 4 confirms that AI Math Tutor exceeds the capabilities of both static worksheet distribution and basic online quiz platforms across all evaluated dimensions, including adaptivity, AI tutoring, teacher analytics, and local deployment capability.

Future work will focus on extending the system to support additional mathematical subjects and grade levels, integrating handwritten input recognition for open-ended problem solving, implementing cloud deployment for multi-school access, and incorporating a gamified reward system with badges and leaderboards to further enhance student motivation. Integration with curriculum standards (such as CBSE or State Board syllabi) is also planned to ensure alignment with formal educational requirements.

REFERENCES

1. J. A. Chudziak and A. Kostka, “Multi-agent AI tutoring platforms for adaptive mathematics learning,” *Lecture Notes in Computer Science*, vol. 14200, pp. 112–125, Oct. 2022.
2. N. E. M. Mosia and F. O. Egara, “Systematic review of AI-driven personalized learning in mathematics problem solving,” *Frontiers in Computer Science*, vol. 4, pp. 1–15, Jan. 2023.
3. Y. Chen, N. Ding, H. T. Zheng, Z. Liu, M. Sun, and B. Zhou, “Chaining large language models for mathematics tutoring: Effects on student anxiety,” *Proc. ACM Int. Conf. Information and Knowledge Management (CIKM)*, pp. 245–256, Nov. 2024.
4. S. Gupta and R. Mehta, “Adaptive worksheet generation using AI chatbots for middle school mathematics,” *IEEE Access*, vol. 11, pp. 14532–14544, Feb. 2023.
5. L. Wang and H. Li, “Socratic questioning in AI tutoring agents: A reinforcement learning approach,” *Computers & Education*, vol. 195, pp. 104–118, Apr. 2023.
6. M. Patel and K. Srinivasan, “Impact of AI-based tutoring on mathematical problem-solving skills,” *Education and Information Technologies*, vol. 28, no. 5, pp. 6231–6249, May 2023.
7. A. Kostas and P. Dimitriou, “Reducing math anxiety through AI-integrated teacher training programs,” *Journal of Educational Technology & Society*, vol. 26, no. 3, pp. 77–89, Jul. 2023.
8. R. Kumar and J. Singh, “AI-enabled dashboards for student progress tracking in mathematics,” *IEEE Transactions on Learning Technologies*, vol. 16, no. 4, pp. 512–523, Oct. 2023.
9. F. Zhang and Y. Zhao, “Adaptive feedback mechanisms in AI math tutors,” *International Journal of Artificial Intelligence in Education*, vol. 33, no. 2, pp. 201–220, Mar. 2024.
10. H. Tanaka and M. Saito, “Project-based learning supported by AI tutors: A case study in advanced mathematics,” *British Journal of Educational Technology*, vol. 55, no. 1, pp. 45–61, Jan. 2024.

11. P. R. Sharma and A. Verma, “AI-driven hint generation for problem-solving worksheets,” *Interactive Learning Environments*, vol. 32, no. 6, pp. 923–940, Jun. 2024.
12. J. Lopez and C. Martinez, “Data privacy in AI-enabled math tutoring systems,” *IEEE Transactions on Education*, vol. 67, no. 2, pp. 134–142, Apr. 2024.
13. K. Müller and T. Becker, “Evaluating student behavior logs in AI tutoring systems,” *Computers in Human Behavior*, vol. 145, pp. 107–119, Aug. 2024.
14. S. Ahmed and M. Rahman, “Cloud infrastructure for scalable AI tutoring platforms,” *Future Generation Computer Systems*, vol. 152, pp. 321–335, Sep. 2024.
15. D. Park and J. Lee, “Integrating AI tutors with traditional classroom methods: A hybrid approach,” *Educational Technology Research and Development*, vol. 72, no. 5, pp. 1123–1140, Oct. 2024.